

Day 4 – Text Encoding, Embedding, and Tokenization

Tuesday, May 12, 2026 5:15 AM

*** RECAP – PREVIOUS SESSION ***

- install VS Code, through UV
- setup the virtual environment

Setting up the Virtual Environment in VS Code.

```
>> uv python list (it lists all the python versions)
>> uv venv env --python <python version>
>> <path> (to activate the virtual environment)
>> uv pip install -r requirements.txt
```

Requirments.txt

```
scikit learn==1.4.2
rank-bm25==0.2.2
numpy==1.26.4
scipy==1.11.4
sentence-transformers==2.7.0
gensim==4.3.2
torch>=2.4.0
torchvision>=0.19.0
transformers>=4.41.0
datasets>=2.19.0
pillow>=10.2.0
mteb
ipkernel
langchain_google_genai
langchain-openai
```

*** VECTOR ***

- Vector ---> It is a set of Numerical Value. Whenever, we represent data in DL (Deep Learning) or GenAI, it is always represented in the form of Vector or Matrix.
 - 5 is a Scalar Value (a single value)
 - [5, 9] is a Vector (a collection of value)
 - [[2, 4], [5, 3]] is a Matrix (a grid of number – a collection of vector). Matrix is a collection of Vector (when looked from either the row end or the column end).
 - [[1, 4], [4, 6]] is a 2 * 2 matrix (row, column)
 - [[1, 2], [2, 5], [5, 8]] is a 3 * 2 matrix (row, column)
 - [[1, 2, 3], [3, 4, 5]] is a 2 * 3 matrix (row, column)
- A vector is a numerical representation of data in n-dimensional space.
 - [1, 2, 3, 4] ---> data is represented in 4 dimensions.
 - [1, 2, 3, 4, 5, 6] ----> data is represented in 6 dimensions.
- A vector is a mathematical term that is represented by the magnitude + dimension.
- In simple terms, vector is a collection of numbers.
- Both Encoding and Embedding are represented in the form of vector.
 - ENCODING ---> numerical representation of data (count based method ---> frequency based)
 - EMBEDDING ---> meaningful numerical representation of data (neural network or transformer based method)

*** DEEP-DIVE INTO VECTOR ***

- Vector has magnitude and direction.
- n-dimension of a vector [7, 2, 1, 4, 6,, n]
- Magnitude is the distance between the origin and the point. It is called using Pythagoras Theorem (Euclidean Distance).
- Euclidean distance is used in Similarity Search.
- --- Similarity Concept ---

Similarity between iPhone & Apple is Zero.

iPhone (Apple) vs Apple

	iPhone	Apple
Technology	1	0
Fruit	0	1

iPhone = [1, 0]

Apple= [0, 1]

$\text{DOT PRODUCT} = (x_1, y_1) (x_2, y_2) = (x_1 * x_2) + (y_1 * y_2)$

$\text{DOT PRODUCT} = (1, 0) (0, 1) = (0, 0)$

Therefore, mathematically we have been that the similarity between the two vectors iPhone and Fruit is Zero.

Feature 1 (f1) = Fruit

Feature 2 (f2) = Technology

Angle between the two vector F1 & F2 = 90

Cosine between Feature 1 & Feature 2 => $\cos 90 = 0$

Therefore, mathematically we have been that the similarity between the two vectors iPhone and Fruit is Zero.

*** ENCODING ***

- Definition: Converting the raw data (text) into numerical form (often sparse vector) without necessarily preserving the semantic meaning.
 - sparse vector is a ZERO vector
- It is a count based method, where we check the frequency of the word and create a vector for that.
- It is a well know method for -
 - One-Hot Encoding
 - BOW (Bag-of-Words)
 - TF-IDF (Term Frequency – Inverse Document Frequency)
 - N-Grams
 - BM25 - improved extension of TF-IDF, but technically it is a ranking algorithm and not just an encoding method. It will be illustrated further in the RAG chapter where the practical use will be discussed.
 - GloVe (Global Vectors) is a count-based + matrix factorization based.

*** DEEP-DIVE INTO ENCODING ***

- Encoding is a way to convert the text data into a vector (n-dimensional vector).
- The techniques is called the count based technique -
 - One-Hot Encoding
 - Bag-of-Words (BOW)
 - TF-IDF
 - BM-25 (built on top of TF-IDF, and is a ranking based method)
 - GloVe vector (not used now-a-days)

*** EMBEDDING ***

- Embedding is mathematical representation of the data.
- Embedding is a vector with meaningful numbers, that captures semantic relationship.
- Embedding is a way to covert text, image, audio, video into meaningful numbers.
- Embedding is dense, high-dimensional numerical vector that represents the semantic meaning of data.
 - Dense: Non Spars (*will not have more zeros in the embedding*)
 - High-Dimensional: Typically ranges from 100 to several thousand dimensions depending on model. 384, 768, 1024, 1536, 2048, etc....
 - Vector: Mathematical object in linear algebra in programming this vector is typically stored as an array, means a collection of numerical values (float).
- Example -

Converts data into a numerical vector that preserves meaning.

 - Embedding("king") vs Embedding("queen")
 - distance between these embedding (or) the angle between these embedding would be less.
 - Embedding("king") vs Embedding("banana")
 - distance between these embedding (or) the angle between these embedding would be huge.
- There are three ways to check the similarity -
 - Dot Product
 - Cosine Similarity
 - Euclidean Distance (ED)
- Classical Word Embedding Models (based on Neural Network)
 - Word2vec
 - fasttext
- Limitations:
 - They create embedding at the word level only.
 - They produce static embeddings (fixed vector).
 - They do not understand the context.

*** DEEP-DIVE INTO EMBEDDING ***

- Embedding is a way to convert the data (text, image, video, audio) into a meaningful vector.
- It is not a count based method, and uses neural networks.
- Now a days, all the embedding models in the market are using transformer based method, and the technique that is used is called SOTA (State-of-Art).

- Corpus ---> collection of documents
- Documents ---> collection of paragraphs
- Paragraph ---> collection of sentences
- Sentences ---> collection of words/tokens
- Word ---> may split into tokens (collection of characters)
- Token ---> model input unit (collection of characters)
- Character ---> smallest or fundamental unit of the data (text)
- The early stage (2018 - 20) embedding models (**word2vec, fasttext, and others**) are able to convert the data (word) into embedding and are based on **neural network architecture**. This model supports **only text type of data**.
- In today's world, the latest embedding model (**SOTA**) is able to convert the data (corpus, documents, paragraphs, sentences, or word) into embedding and is based on **transformer architecture**. This model supports **text, image, audio, and video type of data**.
- BERT, MP-Net, BGE, Gemini, OpenAI, Claude ----> all of these use dynamic embedding (SOTA embedding model).
- Example -
 - Sentence 1: I sat on the river bank.
 - Sentence 2: I deposited money in the bank.
 - Word: bank (the context/meaning of the word "bank" is different in each of the above sentence).

In a **static embedding model** (word2vec), the word "bank" is represented by the same vector in both sentences. The context of the sentence is not taking into account in a static embedding model.

bank ---> [0.21, -0.44, 0.89, ...]

In **dynamic embedding model** (SOTA transformer-based), the word "bank" will be represented by the different vector, because **SOTA model understands the context of the sentence**.

bank ---> [0.21, -0.44, 0.89, ...]

bank ---> [0.93, 0.01, 0.75, ...]

- Open-Source Models & Closed-Source Models for both **static & dynamic embedding model**.

*** ENCODING vs EMBEDDING ***

	Encoding	Embedding
Purpose	Convert data to numbers	Convert data to numbers (represent meaning)
Nature	Often Sparse (frequency or count based)	Dense (Neural Network or Transformer based)
Semantic Meaning	Not Preserved	Preserved
Context Awareness	Not Preserved	Preserved

*** SEMANTIC SEARCH ***

Data ---> Embedding/Vector ---> to check what are the other similar vectors, we have a technique called as semantic search (dot product, euclidean distance, cosine similarity)

Semantic Search / Similarity Search ---> based on the context of the given vector, other similar vectors are searched.

- What is a semantic search (dot product and cosine similarities)
- Semantic Search ---> meaning-based intelligent search using embedding. It is also called the similarity search.
- Semantic Search ---> searching based on meaning of the data, and not just extract words. It uses the transformer model, where the transformer model (vector) is capable of to preserve the meaning of the data.
- Semantic search is usually performed using techniques -
 - Cosine Similarity
 - Dot Product
 - Euclidean

Example #1-

- Sentence 1: Eating fiber reduces heart risk.
- Sentence 2: Eating fruits and vegetables lowers cardiovascular disease chances.

Both sentences have the same meaning.

Even though the words are different, embedding place them close together in vector space.

When a vector is generated out of the above two sentences, it is going to show a very high similarity / it going to be a very high similar vector.

That's why semantic search returns relevant results even when wording changes.

Example #2-

- Sentence 1: Eating fiber reduces heart risk
- Sentence 2: Buying a new car improves driving comfort.

Both sentences have completely different meaning, and show completely different topics.

When a vector is generated out of the above two sentences, it is going to show a very low similarity (mathematically).

It also shows low cosine similarity.

*** DIFFERENCE BETWEEN KEYWORD SEARCH & SEMANTIC SEARCH ***

Keyword Search ---> Keyword search matches the word.

Semantic Search ---> Semantic search matches the meaning. It finds out similarity using matrix like cosine, euclidian distance, dot product, etc.

How keyword-based search work -

- If you search: "How to reduce heart risk" (Sentence 1)
The system will mainly look for sentence containing those exact words like -
 - Reduce
 - Heart
 - Risk
- If a sentence instead contains: "Ways to lower cardiovascular disease chances" (Sentence 2)
- A traditional keyword search may not match it because:
 - reduce != lower
 - heart risk != cardiovascular disease
- Even though the meaning is similar, the words are different (i.e., Sentence 1 and Sentence 2 are semantically same).

Example -

Sentence 1: how to lose weight fast

Sentence 2: quick fat loss strategies

Keywords Search ---> weak match (always works with the keyword)

Semantic Search ---> strong match (always works with the semantic meaning/context of the vector → this is captured using transformer based model → the vector generated here is a meaningful vector)

*** APPLICATION OF EMBEDDING ***

1. Semantic Search (Most Fundamental) - meaning based search instead of matching keywords, based on vector similarity
2. Google like search engines
3. Google Image search
4. Pinterest – user searches "a red car" and the system retrieves the "images of red cars"
5. Recommendation Systems -
 - a. Netflix – similar movies
 - b. Amazon – similar products
 - c. Spotify – similar songs
6. Topic Modeling (Clustering)
7. Group similar documents together – News Categorization
8. RAG (Retrieval-Argument Generation) ---> the retrieval process in RAG method is completely dependent on semantic/similarity search and it is the biggest application of embedding concept.

Data ---> numerical vector ---> find similarity with the vector

*** MTEB (MESSAGE TEXT EMBEDDING BENCHMARK) LEADERBOARD ***

MTEB will help us to check which embedding model is a good one vs which embedding model is not a good one.

<https://huggingface.co/spaces/mteb/leaderboard>

<https://github.com/embedding-benchmark/results>

<https://sbnet.net/> (Sentence Transformer Repository)

<https://huggingface.co/models?library=sentence-transformers>

*** MATHEMATICAL INTUTION FOR ENCODING & EMBEDDING ***

- Encoding Methods
 - One-Hot Encoding
 - BOW (Bag-of-Words)
 - TFIDF
- Embedding
 - Embedding using Neural Network (Classical Method)
 - Word2Vec
 - Embedding using Transformer Architecture
 - SOTA – using SOTA we can convert any text (sentence, document, or book) into word/text/image/video/audio

***** Summary *****

1. Vectors
 - o Dot Product
 - o Cosine Similarity
 - o Euclidian Distance
 2. Encoding vs Embedding
 3. Semantic Search (Similarity Search) vs Keyword Search
 4. Application of the Embedding
-
-
-

For any questions - snsrivas3365@gmail.com